Tema 2 — Modelos de Regresión

Resumen práctico y de programación

Certificación Profesional en Ciencia de Datos (IBM)

Índice

1. Introducción general **2**

2. Regresión Lineal **2**
- 2.1. Regresión lineal simple . . . 2
  - 2.1.1. Código típico . . . 2
  - 2.1.2. Conceptos clave de programación . . . 2
  - 2.1.3. Cuándo usarla . . . 3
- 2.2. Regresión Lineal Múltiple . . . 3
  - 2.2.1. Código típico . . . 3
  - 2.2.2. Diferencias con la simple . . . 3
  - 2.2.3. Estandarización . . . 3
  - 2.2.4. Interpretación de coeficientes . . . 3

3. Métricas de Regresión **4**
- 3.1. MAE (Mean Absolute Error) . . . 4
- 3.2. MSE (Mean Squared Error) . . . 4
- 3.3. RMSE (Root Mean Squared Error) . . . 4
- 3.4. R^2 (Coeficiente de determinación) . . . 4

4. Regresión Logística **4**
- 4.1. Modelo . . . 4
- 4.2. Código típico . . . 4
- 4.3. Diferencia clave . . . 4
- 4.4. Estandarización en logística . . . 5
- 4.5. Coeficientes . . . 5

5. Métrica principal en Logística **5**
- 5.1. Log Loss . . . 5

6. Aplicaciones reales **5**
- 6.1. Regresión lineal . . . 5
- 6.2. Regresión logística . . . 5

7. Guía rápida de elección **5**

1. Introducción general

En este tema se estudian los primeros modelos fundamentales de Machine Learning supervisado:

- Regresión lineal simple
- Regresión lineal múltiple
- Regresión logística

Todos estos modelos comparten una estructura común:

- Datos etiquetados
- Separación en **features (X)** y **target (y)**
- Entrenamiento, predicción y evaluación

La diferencia principal está en **qué tipo de variable se predice** y **cómo se interpreta el resultado**.

2. Regresión Lineal

La regresión lineal se utiliza para predecir un **valor continuo**.

2.1. Regresión lineal simple

Modelo matemático:

$$\hat{y} = b_0 + b_1 x$$

- x: una única variable explicativa
- b_0: intercepto
- b_1: pendiente (impacto de x sobre y)

2.1.1. Código típico

```
X = cdf['ENGINESIZE'].to_numpy()
y = cdf['CO2EMISSIONS'].to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train.reshape(-1,1), y_train)
```

2.1.2. Conceptos clave de programación

- **to_numpy()**: convierte Series/DataFrame de pandas a arrays NumPy
- **reshape(-1,1)**: convierte un vector 1D en matriz 2D
- **fit()**: entrena el modelo
- **coef_**: coeficiente b_1
- **intercept_**: término independiente b_0

2.1.3. Cuándo usarla

- Relación aproximadamente lineal
- Interpretabilidad sencilla
- Análisis rápido de impacto de una variable

2.2. Regresión Lineal Múltiple

Modelo matemático:

$$\hat{y} = b_0 + b_1 x_1 + b_2 x_2 + \cdots + b_n x_n$$

- Se usan varias features simultáneamente
- Cada coeficiente representa el impacto medio de una variable

2.2.1. Código típico

```
X = cdf[['ENGINESIZE','CYLINDERS','FUELCONSUMPTION_COMB']]
y = cdf['CO2EMISSIONS']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
```

2.2.2. Diferencias con la simple

- X ya es una matriz 2D $\rightarrow$ no se necesita reshape
- Hay un coeficiente por feature
- Se pueden explicar subgrupos de datos

2.2.3. Estandarización

En muchos casos se usa **StandardScaler**:

$$x' = \frac{x - \mu}{\sigma}$$

```
scaler = StandardScaler()
X_std = scaler.fit_transform(X)
```

Sirve para:

- Comparar coeficientes
- Mejorar estabilidad numérica

2.2.4. Interpretación de coeficientes

- Signo positivo: la variable aumenta y
- Signo negativo: la variable reduce y
- La magnitud depende de la escala

3. Métricas de Regresión

3.1. MAE (Mean Absolute Error)

$$MAE = \frac{1}{n} \sum |y - \hat{y}|$$

Error medio en unidades reales.

3.2. MSE (Mean Squared Error)

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

Penaliza mucho los errores grandes.

3.3. RMSE (Root Mean Squared Error)

$$RMSE = \sqrt{MSE}$$

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

Vuelve a las unidades originales de y.

3.4. R^2 (Coeficiente de determinación)

Indica cuánta variabilidad explica el modelo.

4. Regresión Logística

Se utiliza para **clasificación**, normalmente binaria.

4.1. Modelo

$$P(y = 1|x) = \sigma(b_0 + b_1 x_1 + \cdots + b_n x_n)$$

donde:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

4.2. Código típico

```
model = LogisticRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)
```

4.3. Diferencia clave

- predict(): devuelve clase (0 o 1)
- predict_proba(): devuelve probabilidades

4.4. Estandarización en logística

Es especialmente importante porque:

- Mejora la convergencia
- Hace justa la regularización

4.5. Coeficientes

- Indican impacto sobre los **log-odds**
- Signo indica si aumenta o reduce la probabilidad

5. Métrica principal en Logística

5.1. Log Loss

$$-\frac{1}{n}\sum[y\log(p)+(1-y)\log(1-p)]$$

- Penaliza mucho predicciones muy seguras pero erróneas
- Cuanto más bajo, mejor

```
log_loss(y_test, y_prob)
```

6. Aplicaciones reales

6.1. Regresión lineal

- Predicción de ventas
- Costes
- Consumo energético
- Forecast financiero

6.2. Regresión logística

- Churn de clientes
- Fraude
- Aprobación de crédito
- Riesgo médico

La salida probabilística permite ajustar decisiones según coste/beneficio.

7. Guía rápida de elección

- Número continuo → Regresión lineal
- Evento sí/no → Regresión logística
- Interpretabilidad alta → Modelos lineales